

Topic 3: NumPy and Pandas (part1)

Igor Razgon
igor.razgon@durham.ac.uk

Content

1. NumPy

- Accessing individual values

- The use of axis parameter

- Filtering

- 2D filtering using `any()`

2. Pandas

- DataFrame instance

- Filtering of columns

- Mean computation

- Filtering of rows

- Sorting

NumPy

NumPy library

- NumPy stands for Numerical Python.
- NumPy arrays provide an alternative way of working with data collections.
- It is claimed that NumPy based applications work quicker than the same applications using ordinary Python lists and dictionaries.

Why are we considering NumPy?

- Pandas (our main object of study) is based on NumPy.
- Pandas and NumPy share several features that may look outlandish for people with experience in ordinary Python.
- It is easier to first consider these features in a 'minimalist' environment (that is, in NumPy).

Importing NumPy

- For importing NumPy, 2e will use the command
import numpy as np
at the beginning of the notebook.
- The command will not be explicitly mentioned in the subsequent fragments of code shown on the slides.

Approaching individual elements of a *one dimensional* Numpy array

```
arr = np.array([1, 2, 3])
```

#The indexing here is exactly as in the ordinary lists.

```
arr[1]
```

- The output of the fragment is
2
- The last line is a way to provide output in a Jupyter notebook. It will not work in an ordinary Python.

Approaching individual elements of a bidimensional NumPy array

```
arr2d = np.array([[1,2,3,4,5], [6,7,8,9,10]])
```

#Note the difference: for ordinary lists,

#the reference would have been arr2d[1][0]

```
arr2d[1,0]
```

- The output of the fragment is

6

The axis parameter

- The use of *axis* parameter is abundant in both NumPy and Pandas.
- We specify *axis=0* or *axis=1* depending whether we want to apply the considered operation to rows or to columns.
- Which of these settings (*axis=0* or *axis=1*) means rows and which columns? It needs to be checked for the specific function by experimenting.
- We will first consider the *axis* parameter for the function ***concatenate***.

Two ways of concatenating two *bidimensional* arrays

- Suppose we want to concatenate two arrays *arr1* and *arr2*.
- We have two options:
 - option (1):** We can append the rows of *arr2* to the rows of *arr1*
 - option (2):** can append the columns of *arr2* to the end of the columns of *arr1*
- Both options can be implemented by the function ***concatenate***:
 - option (1):** with axis=0 (rows)
 - option (2):** with axis=1 (columns)
- **Important:** the concatenated components must be of the same length. Otherwise, Python will throw an exception.

Example: two ways of concatenation

- Consider the arrays

```
arr1=np.array([[1,2],[3,4]])
```

```
arr2=np.array([[5,6],[7,8]])
```

- First concatenate them along the rows

```
arrconc0=np.concatenate((arr1,arr2),axis=0)
```

```
arrconc0
```

with the output `array([[1, 2],[3, 4],[5, 6],[7, 8]])`

- Then along the columns

```
arrconc1=np.concatenate((arr1,arr2),axis=1)
```

```
arrconc1
```

with the output `array([[1, 2, 5, 6],[3, 4, 7, 8]])`

Example: concatenation of two arrays with distinct numbers of columns

- **Consider the arrays**

```
arr3=np.array([[10,20,30],[40,50,60]])
```

```
arr2=np.array([[5,6],[7,8]])
```

- **These arrays can be concatenated only by columns**

```
arrconc3=np.concatenate((arr3,arr2),axis=1)
```

```
arrconc3
```

with the output

```
array([[10, 20, 30,  5,  6],[40, 50, 60,  7,  8]])
```

Concatenation along axis 2

- While we will not need it later, it is interesting to note that *axis* may have values larger than 1.
- Consider an example of concatenation of two 3D arrays with *axis*=2
- The arrays are

```
arr3d1=np.array([[[1,2],[3,4]],[[5,6],[7,8]]])
```

```
arr3d2=np.array([[[9,10],[11,12]],[[13,14],[15,16]]])
```

- We concatenate them along *axis*=2 to obtain the output

```
array([[[ 1,  2,  9, 10],[ 3,  4, 11, 12],  
       [[ 5,  6, 13, 14],[ 7,  8, 15, 16]]])
```

Filtering using patterns

- NumPy arrays with Boolean values (*patterns*) have a special role: they can be used for filtering of other NumPy arrays.
- For example,

#Array to be filtered

```
arr=np.array([10,20,-5,7,-6,2])
```

#Filtering mask

```
pattern1=np.array([True,False,True,False,False,False])
```

#Applying the filter: only elements corresponding to True entries remain

```
arr1=arr[pattern1]
```

```
arr1
```

The output is `array([10, -5])`

Creating patterns using conditions

- In the previous example, the Boolean mask for filtering was initialized independently of the input array.
- An important feature of NumPy is the possibility to initialize the mask as the output of a condition applied componentwise to the input array.

```
arr=np.array([10,20,-5,7,-6,2])
```

```
#pattern2 is the result if applying the >0 condition componentwisely to arr2
```

```
#The syntax is not correct in the ordinary Python
```

```
pattern2=arr>0
```

```
arr2=arr[pattern2]
```

```
(pattern2, arr2)
```

The output is: `(array([ True, True, False, True, False, True]), array([10, 20, 7, 2]))`

Compound filters

Operations *logical_and*, *logical_or*, and *logical_not* can be used to create patterns for compound conditions.

```
arr=np.array([10,20,-5,7,-6,2])
```

```
pattern2=arr>0
```

```
pattern3=arr<20
```

#The True elements of ptand corresponds to the entireties of aaa that are greater than 0 and smaller than 20

```
ptand=np.logical_and(pattern2,pattern3)
```

```
arr3=arr[ptand]
```

```
arr3
```

The output is: `array([10, 7, 2])`

Filtering for 2D arrays

- Filtering of 2D arrays (tables) is an important task in Data Analytics.
- For example, if rows correspond to employees and one of the columns (attributes) is the DOB then we may apply filtering to consider only the employees born within 1970 and 1995.
- We will consider this kind of task when we study Pandas.
- For NumPy, we will now have a look at filtering approaches involving conditions with **quantifiers**.
- This means that we leave only rows (or columns) where either
 - At least one value matches a specific condition or
 - Or the values match the specific condition.
- This filtering is implemented using operations *np.any* and *np.all*

Bidimensional patterns. Operation *np.any()*

- A *bidimensional pattern* is just a 2D NumPy array with all values being *Boolean*.
- Operation *any* applied to a bidimensional pattern with *axis=1* returns a Boolean mask for the rows with *False* entries corresponding to the “**all *False***” rows.
- Operation *any* applied to a bidimensional pattern with *axis=0* returns a Boolean mask for the columns with *False* entries corresponding “**all *False***” columns.
- **Important:** note the difference from ***concatenate***:
 - *axis=1* corresponds to rows
 - *axis=0* corresponds to columns.

Bidimensional patterns. Operation *np.all()*

- A *bidimensional pattern* is just a 2D NumPy array with all values being *Boolean*.
- Operation *any* applied to a bidimensional pattern with *axis=1* returns a Boolean mask for the rows with *True* entries corresponding to the “**all *True***” rows.
- Operation *any* applied to a bidimensional pattern with *axis=0* returns a Boolean mask for the columns with *True* entries corresponding to the “**all *True***” columns.
- **Important:** note the difference from *concatenate*:
 - *axis=1* corresponds to rows
 - *axis=0* corresponds to columns.

Testing *np.any()*

```
pattern=np.array([[False, True, True],[False, True, False],[False, False, False]])
```

#Forming the Boolean mask for the rows. False entries correspond to all Falses

```
rowany=np.any(pattern,axis=1)
```

```
rowany
```

The output is: *array([True, True, False])*

#Filtering the Boolean mask for the columns. False entries correspond to all Falses

```
colany=np.any(pattern,axis=0)
```

```
colany
```

The output is: *array([False, True, True])*

Testing *np.all()*

#Only rows with all True entries correspond to True in the mask

rowall=np.all(pattern,axis=1)

rowall

It turns out that there are none: *array([False, False, False])*

Removal of 'all negative' rows using *np.any*

```
arr=np.array([[15,-1,2],[6,-1,0],[-3,-4,-2]])
```

```
#Creating a Boolean mask for non-negative entries
```

```
pattern1=arr>=0
```

```
pattern1
```

The mask is: `array([[ True, False, True], [ True, False, True], [False, False, False]])`

```
#Removal of all negative rows using np.any
```

```
arrpos=arr[np.any(pattern1,axis=1)]
```

```
arrpos
```

The output is: `array([[15, -1, 2],[ 6, -1, 0]])`

Pandas and DataFrame

- ***Pandas*** is the main Python package for data analytics.
- Contains a huge number of descriptive statistics operations.
- Most of them can be easily implemented in the ordinary Python 😊 but what makes Pandas special is the connection to visualization libraries.
- The main class for storing datasets is called ***DataFrame***.
- We are going to consider the class in great detail.

np.delete : a generic filtering tool

- The construct `arr1=arr[pattern]` where `arr` is a 2D array and `pattern` is a one-dimensional Boolean array corresponding to the rows is a convenient tool for rows filtering.
- If we need filtering of columns of a NumPy array, there is a generic function `np.delete` (depending on the specified axis, it can also be used for removal of rows).
- You are expected to familiarise yourself with the use of `np.delete` for filtering of rows and columns using one dimensional Boolean masks.
- Corresponding tasks during practicals as well as self control questions will be provided for your support.
- **There are questions related to `np.delete` in the question bank for the benchmark**

Pandas

Importing Pandas

- In order to use Pandas, we write the line
import pandas as pd
at the beginning of the file
- In the fragments of code we are to consider next, we will assume the line to be included without explicitly mentioning it.

Data frames vs. dictionaries

- The main class of Pandas is called *DataFrame*.
- *DataFrame* instance can be thought as a table with enumerated rows and named columns.
- A new *DataFrame* instance can be created from a Python *dictionary*.
- Each dictionary key must be associated with a list, all lists must be of the same length.
- The lists become the columns, the corresponding keys being the column names.

DataFrame instance created from a Python dictionary

#The dictionary to be turned into a DataFrame

```
marks={  
    'id': [11,41,52,8],  
    'algebra':[60,90,70,55],  
    'analysis':[40,80,80,70],  
    'logics':[90,80,90,80]  
}
```

#DataFrame initialization

```
pdmarks=pd.DataFrame(marks)
```

pdmarks

The resulting *DataFrame* instance is:

	id	algebra	analysis	logics
0	11	60	40	90
1	41	90	80	80
2	52	70	80	90
3	8	55	70	80

Restriction of columns

- A new *DataFrame* instance can be obtained from existing one by removal of columns.
- This is important for various per row statistics:
we leave only the columns with values meaningful for the statistics.
- One way to it is as follows:
 - Create a list L with the names of columns to stay in the new DataFrame.
 - Let df be the DataFrame being restricted.
 - Create a DataFrame df_{new} by running $df_{new}=df[L]$.

Removal student IDs from the *pdmarks* data frame

#getting ird of IDs (may be useful for computinng averages)

marksonly=['algebra','analysis','logics']

pdmarksonly=pdmarks[marksonly]

pdmarksonly

The resulting *DataFrame* instance is:

	algebra	analysis	logics
0	60	40	90
1	90	80	80
2	70	80	90
3	55	70	80

Descriptive Statistics in Python Pandas

Sr.No.	Function	Description
1	count()	Number of non-null observations
2	sum()	Sum of values
3	mean()	Mean of Values
4	median()	Median of Values
5	mode()	Mode of values
6	std()	Standard Deviation of the Values
7	min()	Minimum Value
8	max()	Maximum Value
9	abs()	Absolute Value
10	prod()	Product of Values
11	cumsum()	Cumulative Sum
12	cumprod()	Cumulative Product

Mean computation

- Let *df* be a *DataFrame* instance.
- *df.mean()* returns averages of each column and *df.mean(1)* returns averages of each row.
- The type of the returned object is **Series**, effectively a *1D DataFrame* to be considered later.
- **Important:** *mean()* is *mean(0)* by default, the number in the brackets specifies the axis.
- **Note:** again, *0* corresponds to columns, and *1* corresponds to rows.

Computing average marks of students

#Computing averages.

#Note that the axis 1 of the mean

#mean that the averages are computed per row

#Note also the need to exclude ID column to avoid

#inclusion of IDs into the averages

avmarks=pdmarksonly.mean(1)

avmarks

The output is :

0	63.333333
1	83.333333
2	80.000000
3	68.333333
dtype: float64	

Adding a column to a *DataFrame* instance

- We can easily *add* a new column to a *DataFrame* instance.
- Just write the name of the instance with the name of the new column in the bracket and assign it with a Series instance.
- The new column will become the last one in the new data frame.
- For example, we can add average marks as a new column of the *pdmarks* instance

#Adding the averages as a new column to the initial dataframe

```
pdmarks['Avmarks']=pdmarksonly.mean(1)
```

<i>pdmarks</i>	id	algebra	analysis	logics	Avmarks
0	11	60	40	90	63.333333
1	41	90	80	80	83.333333
2	52	70	80	90	80.000000
3	8	55	70	80	68.333333

Sorting rows by a single column

- Apply function *sort_value*.
- To specify the column, use parameter *by*="Column Name"
- By default, sorting is done in the *ascending* order.
- To sort in the descending order, use *ascending=False*

Sorting student marks by the descending order of their averages

- In a standard end of academic year report student records are sorted by the creasing order of their averages.
- With the averages column added to *pdmarks*, we are now going to apply the sorting operation to *pdmarks*

```
pdavsorted=pdmarks.sort_values(by='Avmarks', ascending=False)
```

pdavsorted

	id	algebra	analysis	logics	Avmarks
1	41	90	80	80	83.333333
2	52	70	80	90	80.000000
3	8	55	70	80	68.333333
0	11	60	40	90	63.333333

Filtering of rows of a *DataFrame* instance

- Similarly to NumPy, if we write the name of a *DataFrame* with a one-dimensional Boolean mask in the square brackets the rows corresponding to the *False* entries of the mask.
- The mask can be created by applying a condition to a single column of a *DataFrame* instance much like we did for one-dimensional NumPy arrays.
- We are going to demonstrate this approach by filtering *pdmarks* so that only the records with a first class mark for analysis are left.

Filtering student records with analysis mark<70

```
marksanalysis=pdmarks['analysis']
```

```
marksanalysis      0    40  
                  1    80  
                  2    80  
                  3    70  
Name: analysis, dtype: int64
```

```
firstanalysis=marksanalysis>=70
```

```
firstanalysis      0    False  
                  1     True  
                  2     True  
                  3     True  
Name: analysis, dtype: bool
```

```
pdmarks[firstanalysis]
```

	id	algebra	analysis	logics
1	41	90	80	80
2	52	70	80	90
3	8	55	70	80